

PLUGIN PERFORMANCE ANALYSIS

UltimateLeaderboard v3.1.0 by Mevent

PulseRust — Sydney High-Performance Modded Server

9800x3D · 96 GB RAM · Dual NVMe · 10 Gb Network · High-Pop Modded Environment

FINDINGS	CRITICAL	HIGH	MEDIUM
10	3	4	3

This report identifies 10 issues in UltimateLeaderboard that contribute to main-thread stalls on a high-population modded Rust server. The three Critical findings (UL-01, UL-02, UL-03) are the primary contributors to the 100ms+ stall clusters observed under raid and high-event-rate conditions. All findings are ordered from most to least severe impact.

● CRITICAL — Immediate Action Required

CRITICAL

UL-01 — GetEntry() Blocks Main Thread on Stat-Event Cache Miss

Impact: 10–50ms stall
per miss

PROBLEM

- GetEntry() is called inline on every kill, death, gather, and explosive event.
- If the player's .pb file has not yet been processed by the boot-load coroutine, GetEntry() falls through to a synchronous File.Exists + File.ReadAllBytes on the main thread.
- During a raid with 20+ simultaneous kill events, this can trigger 20 back-to-back blocking disk reads — each one holding the main thread until the kernel returns.
- This is the same root cause as the original Kits plugin K-01 finding before it was fixed.

```
// BEFORE – blocks main thread if player not yet in cache
private PlayerStats GetEntry(ulong id) {
    if (!_playerStats.TryGetValue(id, out e)) return e;
    e = File.Exists(path) ? Deserialize(id, File.ReadAllBytes(path)) // BLOCKING
    : new PlayerStats(id);
    _playerStats[id] = e;
    return e;
}

// AFTER – stat hooks use TryGet, never block
private void OnEntityDeath(...) {
    if (!PlayerStats.TryGet(attacker.userID, out var stats)) return; // safe drop
    stats.AddStats(LootType.Kill, entity.ShortPrefabName, 1);
}
```

FIX

- ✓ Guard all stat-event paths with TryGet() (read-only) — silently drop or queue updates for players whose data is not yet loaded.
- ✓ Introduce a pending-update queue: buffer stat increments keyed by userId, replay them in OnRead() once the player file finishes loading from the worker thread.
- ✓ Never call File.ReadAllBytes on the main thread from a stat hook.

CRITICAL

UL-02 — LoadAllClans() Fully Synchronous on Startup / Reload

Impact: 50–200ms stall
at boot

PROBLEM

- LoadAllClans() is called from LoadDataStorage() which runs on the main thread during OnServerInitialized.
- It calls Directory.GetFiles() then File.ReadAllBytes() for every clan file in sequence — no yielding, no async, no batching.
- A server with 50+ active clans reads 50+ files synchronously before the server resumes ticking. Any plugin reload mid-session (e.g. Oxide reload after config change) repeats this stall.
- Players connecting or actions firing during this window experience lag as the main thread is blocked in the filesystem.

```
// AFTER — batched clan load, non-blocking
private IEnumerator LoadAllClansRoutine(string[] files) {
    var processed = 0;
    for (var i = 0; i < files.Length; i++) {
        var bytes = File.ReadAllBytes(files[i]); // still on main thread per frame
        var clan = DeserializeClan(bytes);
        if (clan != null) _clanStats[clan.ClanTag] = clan;
        if (++processed < BOOT_LOAD_BATCH) continue;
        processed = 0;
        yield return null; // release main thread for one frame
    }
    _clansLoaded = true;
}
```

FIX

- ✓ Move clan loading into a coroutine using the same batched pattern as LoadAllPlayersRoutine.
- ✓ Process BOOT_LOAD_BATCH clan files per frame, yielding between batches.
- ✓ Set a _clansLoaded flag on completion; guard any clan stat writes until flag is set.
- ✓ Alternatively, hand clan reads to the existing PlayerSaveWorker via IsWrite=false jobs and assemble _clanStats via NextTick callbacks.

CRITICAL

UL-03 — All Online Players Marked Dirty Every 10 Seconds → Mass Serialization on Save

Impact: 60 serializations
per OnServerSave

PROBLEM

- `SendPlayersTimeUpdate()` fires every `RealTimeStatsUpdateInterval` (default 10s) AND is called explicitly from `OnServerSave()` — meaning it fires twice in close succession every save cycle.
- It sets `stats.Dirty = true` for every online player unconditionally, regardless of whether their playtime delta is meaningful.
- `SavePlayers()` then calls `Serialize()` on every dirty player on the main thread before handing bytes to the background worker — 60 protobuf serializations in one synchronous burst on a full server.
- Combined with 6 sibling plugins doing the same pattern in `OnServerSave`, the total serialization cost per save frame compounds significantly.

```
// BEFORE — serializes ALL online players on main thread per OnServerSave
private void OnServerSave() {
    SendPlayersTimeUpdate(); // marks 60 players dirty
    ServerMgr.Instance.Invoke(SaveAll, Random.Range(5f, 25f));
}

// AFTER — separate concerns, move serialization off main thread
private void OnServerSave() {
    ServerMgr.Instance.Invoke(SaveAll, Random.Range(5f, 25f));
}

private void SavePlayers() {
    foreach (var kv in _playerStats) {
        if (!kv.Value.Dirty) continue;
        kv.Value.Dirty = false;
        var snapshot = kv.Value; // capture reference
        _worker.Enqueue(new Job { Serialize = () => Serialize(kv.Key, snapshot) });
    }
}
```

FIX

- ✓ Remove the explicit `SendPlayersTimeUpdate()` call from `OnServerSave()` — the periodic timer already handles it.
- ✓ Move `Serialize()` calls into the worker thread: enqueue a 'serialize and write' job that receives the `PlayerStats` reference and serializes off-thread.
- ✓ Consider only marking dirty on actual stat changes and disconnect, not on every 10-second time accumulation tick — batch time into an in-memory accumulator and flush only at save.
- ✓ Add a minimum dirty threshold: only serialize if `TotalPlayTime` delta > 30s since last write.

● HIGH — Fix Before Next Wipe

HIGH

UL-04 — ClanFileName() Allocates StringBuilder + N Strings Per Clan Per Save

Impact: GC pressure
each save tick

PROBLEM

- ClanFileName() allocates a new byte[], a new StringBuilder, N intermediate hex strings via bytes[i].ToString("x2"), and a final result string on every call.
- Called during SaveClans() for every dirty clan on each OnServerSave tick.
- With 50 clans and 6-character tags, this is ~300 short-lived string allocations per save contributing directly to Gen0/Gen1 GC pressure.

```
// AFTER — compute filename once at clan creation, reuse forever
public class ClanStats {
    public string ClanTag;
    public string FileName; // set once in GetOrCreate()
    ...
}

public static ClanStats GetOrCreate(string tag) {
    if (!_instance._clanStats.TryGetValue(tag, out var s)) return s;
    s = new ClanStats { ClanTag = tag, FileName = ComputeFileName(tag) };
    _instance._clanStats[tag] = s;
    return s;
}
```

FIX

- ✓ Cache the hex filename string in ClanStats at creation time: compute it once in ClanStats.GetOrCreate() and store as ClanStats.FileName.
- ✓ Or use a pre-allocated char[] buffer with manual hex encoding to avoid all intermediate allocations.
- ✓ Alternatively, hash the clan tag to a ulong and use that as the filename — eliminates the hex encoding entirely.

HIGH

UL-05 — RebuildCache() Snapshots Entire Player Dataset on the Main Thread

Impact: Single large stall
every 10 min

PROBLEM

- RebuildCache() allocates a List and iterates all of _playerStats on the main thread, calling GetRawStatValue() (which runs stat resolution logic) for every player per leaderboard column.
- On a 500-player dataset with 6 columns this is 3,000 stat resolution calls + 500 float[] allocations in one synchronous burst before the job is handed off to the background worker.
- BumpTableCache() calls RebuildCache() after every cache publish, meaning this fires after every leaderboard refresh cycle — potentially every 10 minutes in steady state, or more frequently if external events trigger RebuildCache() directly.

```
// AFTER – snapshot raw values on main thread (fast copy), sort off-thread
public void RebuildCache() {
    if (_isCaching) return; // already in flight
    _isCaching = true;
    // Copy only primitive float[] arrays – fast, minimal allocation
    var snap = _playerStats.Values
        .Where(s => !s.HiddenFromLeaderboard)
        .Select(s => (s.UserId, s.LastName, (float[])s._cachedValues.Clone()))
        .ToArray();
    EnqueueBuild(new CacheBuildInput { PlayerSnaps = snap, ... });
}
```

FIX

- ✓ The snapshot loop itself should run on the worker thread. The main thread should only take a shallow copy of the _playerStats dictionary reference (or snapshot the values array under a brief lock) and pass it to the worker.
- ✓ Use a reader-writer approach: the worker receives an array of (userId, statValues) structs — the main thread copies only the raw float arrays, not the full PlayerStats object graph.
- ✓ Gate BumpTableCache() so it does not re-trigger a rebuild if one is already in flight (_isCaching check before calling RebuildCache).

HIGH

UL-06 — `_colorCache` Dictionary Grows Unbounded Across Session Lifetime

Impact: Memory leak, unbounded heap growth

PROBLEM

- `_colorCache` is keyed by (string colorHex, float alpha) with no maximum size, no eviction policy, and no wipe on reload.
- If leaderboard rank positions or score percentages drive alpha values dynamically, the cache accumulates a unique entry per distinct alpha value ever encountered.
- On a wipe with 500 players cycling through rank positions, hundreds of distinct alpha values can be generated — all permanently resident in the dictionary for the session lifetime.
- This contributes to heap fragmentation and increases the cost of every GC collection as the Gen2 heap grows.

```
// AFTER – quantize alpha to bound cache size
private string GetOrCacheColor(string hex, float alpha) {
    var quantized = (float)Math.Round(alpha * 20f) / 20f; // 0.05 steps
    var key = (hex, quantized);
    if (_colorCache.TryGetValue(key, out var cached)) return cached;
    if (_colorCache.Count > 512) _colorCache.Clear(); // hard cap
    return _colorCache[key] = BuildColorString(hex, quantized);
}
```

FIX

- ✓ Cap `_colorCache` at a reasonable size (e.g. 256 entries) with LRU eviction, or switch to a fixed-size array indexed by quantized alpha (e.g. round to nearest 0.05).
- ✓ Quantizing alpha to 20 distinct values (0.05 steps) bounds the cache to at most `colorHex_count × 20` entries regardless of session length.
- ✓ Clear the cache on plugin reload / wipe to prevent cross-wipe accumulation.

HIGH

UL-07 — GetEntry() Inserts Junk Records for Non-Steam IDs Without Guarding

Impact: Pollutes player dataset and save files

PROBLEM

- GetEntry() inserts a new PlayerStats into _playerStats and persists it if the ID is not already cached, with no IsSteamId() check at the entry point.
- LoadPlayerData() calls GetEntry() directly — if any external plugin passes an NPC userID (0, < 76561197960265728) or server entity ID, an empty record is created and written to disk.
- These junk records are included in every RebuildCache() pass, inflate the cached player count, and show up in API results.
- A corrupted leaderboard caused by a single plugin integration error would require a manual file purge to fix.

```
// AFTER — guard at entry point
private PlayerStats GetEntry(ulong id) {
    if (!id.IsSteamId()) return null; // reject NPCs / server entities
    if (_playerStats.TryGetValue(id, out var e)) return e;
    ...
}
```

FIX

- ✓ Add IsSteamId() validation at the top of GetEntry() — return null (or a sentinel) for any ID that cannot be a valid Steam account.
- ✓ Update all callers to handle a null return from GetEntry().
- ✓ Add the same guard to LoadPlayerData() before it calls GetEntry().

● MEDIUM — Schedule for Next Development Sprint

PROBLEM

- Unity coroutines are not threads — they run on the main thread between yield points.
- `File.ReadAllBytes()` inside the coroutine loop blocks the main thread for the duration of each disk read, even though the coroutine yields between batches.
- With `BOOT_LOAD_BATCH = 50`, each frame processes 50 file reads back-to-back ($50 \times 0.1\text{--}0.5\text{ms} = 5\text{--}25\text{ms}$ stall per frame) during the startup window.
- On a server restarting mid-wipe with 200+ player files, the boot window can stall frames for several seconds total.

```
// AFTER – reads handled by worker thread, zero main-thread blocking
private void LoadAllPlayersAsync(string[] files) {
    _pendingLoads = files.Length;
    foreach (var f in files)
        _worker.Enqueue(new Job { Path = f, IsWrite = false });
    }
    private void OnRead(ulong id, byte[] bytes) {
        if (bytes != null) _playerStats[id] = Deserialize(id, bytes);
        if (--_pendingLoads <= 0) FinishBootLoad();
    }
}
```

FIX

- ✓ Hand file read jobs to `PlayerSaveWorker` via `IsWrite=false` jobs, exactly as Kits v2 does with its `QueueLoad` pattern.
- ✓ The worker reads the file off-thread, then dispatches via `NextTick()` to deserialize on the main thread — minimising blocking time to only the fast deserialization step.
- ✓ The coroutine can be replaced with a simple counter: track pending reads and call `FinishBootLoad()` when the count reaches zero.

MEDIUM

UL-09 — Serialize() Called on Main Thread Before Worker Enqueue

Impact: Compound serialization cost per save

PROBLEM

- SavePlayers() and SaveClans() both call Serialize() / SerializeClan() on the main thread to produce the byte[] payload before enqueueing the job.
- With 60 dirty players per save cycle, 60 BufferStream serialization passes run synchronously on the main thread in the SavePlayers() call.
- While each individual serialization is fast (protobuf binary, pooled stream), the cumulative cost across 60 players adds measurable frame time — especially when combined with the same pattern running in Kits and PulseDailyGems in the same OnServerSave window.

```
// AFTER – serialization runs inside worker thread
private void SavePlayers() {
    foreach (var kv in _playerStats) {
        if (!kv.Value.Dirty) continue;
        kv.Value.Dirty = false;
        var id = kv.Key; var entry = kv.Value;
        _worker.Enqueue(new Job {
            Path = PlayerPath(id),
            Serialize = () => Serialize(id, entry), // runs off-thread
            IsWrite = true
        });
    }
}
```

FIX

- ✓ Restructure Job to carry a Func serializer delegate instead of pre-computed bytes — the worker calls the delegate to serialize off the main thread.
- ✓ Alternatively, pass the PlayerStats reference directly to the worker (under a read lock or snapshot) and let the worker call Serialize().
- ✓ This is a lower-priority change given the worker thread handles the I/O — but eliminates the last remaining main-thread CPU cost from the save path.

MEDIUM

UL-10 — BumpTableCache() Triggers Full UI Rebuild on Every Leaderboard Cache Cycle

Impact: Recurring UI
unavailability every 10
min

PROBLEM

- Every time the background cache worker publishes a new CacheSnapshot, Publish() calls BumpTableCache() which sets _uiTabCacheReady = false and restarts the BuildUiCache() coroutine.
- During the UI rebuild window, any player opening the leaderboard receives a 'Leaderboard is loading, please wait' message — a 10-minute periodic interruption on a busy server.
- If external events (clan wins, manual API calls) trigger RebuildCache() frequently, this window can recur more often than the 600s timer.
- The UI JSON template is structurally static — only the data rows change between rebuilds. The full template does not need rebuilding on every data refresh.

```
// AFTER – data refresh without template invalidation
private void Publish(CacheSnapshot snapshot) {
    _snapshot = snapshot;
    _isCaching = false;
    _instance.isLeaderboardLoaded = true;
    // Do NOT call BumpTableCache() unless columns changed
    // Per-player card values are resolved live via AppendResolvedJson splice
    Interface.Oxide.CallHook("OnUltimateLeaderboardCached");
}
```

FIX

- ✓ Separate the UI template cache (static JSON structure) from the data cache (player rows and sort order). The template only needs rebuilding when the config changes.
- ✓ On leaderboard data refresh, only invalidate the per-player card values — keep _uiTabCacheReady = true and send updated data via the existing AppendResolvedJson splice path without rebuilding the template.
- ✓ Gate BumpTableCache() behind a config-version flag: only trigger a full template rebuild when LeaderboardColumns actually changes.

● Fix Priority Reference Table

ID	Severity	Finding	Primary Fix
UL-01	CRITICAL	GetEntry() blocks on stat-event miss	Guard stat hooks with TryGet(); queue pending updates
UL-02	CRITICAL	LoadAllClans() synchronous on startup	Batch clan load in coroutine; yield between files
UL-03	CRITICAL	60 serializations on main thread per save	Remove duplicate SendPlayersTimeUpdate(); serialize off-thread
UL-04	HIGH	ClanFileName() allocs per save	Cache filename string in ClanStats at creation
UL-05	HIGH	RebuildCache() snapshots on main thread	Move snapshot loop into worker thread
UL-06	HIGH	_colorCache unbounded growth	Quantize alpha; hard-cap at 512; clear on wipe
UL-07	HIGH	GetEntry() inserts junk NPC records	Add IsSteamId() guard at GetEntry() entry point
UL-08	MEDIUM	ReadAllBytes in boot coroutine (main thread)	Hand reads to PlayerSaveWorker; use NextTick callback

UL-09	MEDIUM	Serialize() on main thread before enqueue	Move serialization into worker thread via delegate
UL-10	MEDIUM	UI template rebuilt every 10 min cache cycle	Separate template cache from data cache; skip template rebuild

● Comparison to Kits v2 (Fixed Version)

Capability	Kits v2 (Fixed)	UltimateLeaderboard v3.1.0
Dirty flag pattern	✓ Correct	✓ Correct
Background write thread	✓ Dedicated thread	✓ Dedicated thread
OnServerSave jitter delay	✓ 5–25s random	✓ 5–25s random
Stat-event blocking read	✓ Fixed (TryGet)	✗ Still blocking (GetEntry)
Boot load batching	✓ Async QueueLoad	■ Coroutine but still blocking I/O
Clan data load	N/A	✗ Fully synchronous on startup
Serialize on main thread	■ Minor (fast)	■ Minor (60 players per save)
Memory leak (unbounded dict)	✓ Per-file arch	✗ _colorCache unbounded
Non-Steam ID guard	✓ IsSteamId()	✗ Missing in GetEntry()

UltimateLeaderboard has adopted the correct persistence architecture from Kits v2 (dirty flag, dedicated worker thread, jitter delay). The remaining critical issues are specific to the leaderboard domain: blocking reads on high-frequency stat event paths, synchronous clan boot load, and main-thread serialization volume. Fixing UL-01 through UL-03 will eliminate the 100ms+ stall clusters seen during raid events on this server.